

ChatServlet.java

```
package com.harisudhan.harisudhanmart.controller;

import com.harisudhan.harisudhanmart.service.ChatService;
import com.google.gson.JsonObject;
import java.io.IOException;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
import javax.servlet.ServletConfig;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

/**
 * 04 / Section 11: AI chatbot backend proxy. Validates input, enforces a 10 messages/minute
 * per-session rate limit (Section 17), and never exposes the API key to the client.
 */
@WebServlet("/api/v1/chat")
public class ChatServlet extends BaseServlet {

    private static final int MAX_MESSAGES_PER_MINUTE = 10;
    private static final int MAX_INPUT_LENGTH = 500;

    private ChatService chatService;
    private final Map<String, RateWindow> rateLimits = new ConcurrentHashMap<>();

    @Override
    public void init(ServletConfig config) throws ServletException {
        super.init(config);
        String provider = System.getProperty("ai.chatbot.provider",
            System.getenv().getOrDefault("AI_CHATBOT_PROVIDER", "mock"));
        String apiKey = System.getenv("AI_CHATBOT_API_KEY");
        this.chatService = new ChatService(provider, apiKey);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        HttpSession session = req.getSession(false);
        String sessionId = session.getId();

        if (!allow(sessionId)) {
            writeError(resp, HttpServletResponse.SC_TOO_MANY_REQUESTS, "RATE_LIMITED",
                "Too many messages – please wait a moment before sending another.");
            return;
        }

        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = req.getReader().readLine()) != null) {
            sb.append(line);
        }
        JsonObject body = com.google.gson.JsonParser.parseString(sb.length() == 0 ? "{}" : sb.toString()).getAsJsonObject();
        String message = body.has("message") ? body.get("message").getString() : "";
        if (message.length() > MAX_INPUT_LENGTH) {
            message = message.substring(0, MAX_INPUT_LENGTH);
        }

        String reply = chatService.reply(sessionId, message);
        JsonObject data = new JsonObject();
        data.addProperty("reply", reply);
        writeJson(resp, HttpServletResponse.SC_OK, data);
    }
}
```

```

}

private boolean allow(String sessionId) {
    long now = System.currentTimeMillis();
    RateWindow window = rateLimits.computeIfAbsent(sessionId, k -> new RateWindow(now));
    synchronized (window) {
        if (now - window.windowStart > 60_000) {
            window.windowStart = now;
            window.count.set(0);
        }
        return window.count.incrementAndGet() <= MAX_MESSAGES_PER_MINUTE;
    }
}

private static class RateWindow {
    volatile long windowStart;
    final AtomicInteger count = new AtomicInteger(0);

    RateWindow(long start) {
        this.windowStart = start;
    }
}
}

```